



SIGNLINK

A REAL-TIME PAKISTAN SIGN LANGUAGE TRANSLATION AND SUBTITLING
INTO URDU SYSTEM FOR VIDEO CONFERENCING PLATFORMS

Project Code:

PSLTS-738

Team:

Muhammad Hassan Shahbaz (**BSCS51F22S007**)

Behlol Abbas (**BSCS51F22R038**)

Advisor:

Dr. Fahad Maqbool

Project Manager:

Dr. Muhammad Ilyas

Date: January 7, 2026

Agenda

- ▶ Introduction
- ▶ Problem Statement
- ▶ Project Overview & Objectives
- ▶ Scope: In & Out of Scope
- ▶ System Architecture
- ▶ Technical Design & Components
- ▶ Design Strategies & Considerations
- ▶ Project Plan & Timeline
- ▶ Risks & Mitigation
- ▶ Expected Outcomes & Impact
- ▶ References & Appendices



Introduction

SignLink is an innovative AI-powered accessibility system designed to bridge the communication gap between the Deaf community and hearing participants in virtual meetings. By leveraging real-time computer vision and deep learning, the system recognizes **Pakistan Sign Language (PSL)** gestures from a user's webcam feed, translates them into fluent Urdu text, and generates synchronized Urdu speech and subtitles—all within popular video conferencing platforms like Zoom and Google Meet. Our goal is to enable seamless, inclusive, and natural communication for Deaf and hard-of-hearing individuals, eliminating the reliance on human interpreters and fostering greater participation in academic, professional, and social digital interactions.

Problem Statement

- ▶ **Communication Gap:**

- ▶ Deaf community in Pakistan uses **Pakistan Sign Language (PSL)**.
- ▶ Limited real-time translation tools exist for PSL-to-Urdu.
- ▶ Existing systems only recognize isolated words, not continuous sentences.

Problem Statement Cont...

► **Justification:**

- No continuous PSL translation system exists.
- Global solutions (ASL/BSL) are advanced but not adapted for PSL.
- High demand for inclusive communication in education, workplace, and social settings.

What is SignLink?

- ▶ A real-time **PSL-to-Urdu** translation system for video conferencing.
- ▶ Integrates with platforms like **Zoom** and **Google Meet**.
- ▶ Provides **Urdu subtitles** and **synthesized Urdu speech** for hearing participants.
- ▶ A real time translation that translates at the sentence level.

What are We Doing?

► **Obectives:**

- Real-time gesture recognition using AI/ML.
- Continuous sentence-level translation.
- Plug-and-play desktop application.
- Low-latency processing (<100 ms).



Differentiation & Research Gap

Aspect	Existing PSL Systems	SignLink
Recognition Type	Isolated signs	Continuous sentences
Hardware Needs	Specialized sensors	Standard webcam
Platform Integration	None	Direct SDK integration
Output Modality	Text only	Text + Speech
Processing Location	Often cloud-dependent	Local on-device
Grammar Handling	Word-for-word	Sentence-level with rules

Literature review

Aspect	Global Sign Language Translation (SLT) Research	Pakistan Sign Language (PSL) Research
Research Trend	Rapid evolution from isolated sign recognition to real-time, continuous sentence-level translation	Research remains largely limited and slow to evolve
Primary Capability	Continuous sentence-level sign language translation (video-to-text)	Mostly isolated signs, alphabets, numbers, or word-level recognition
Real-Time Translation	Real-time and low-latency systems exist for multiple sign languages	No real-time PSL-to-Urdu sentence-level translation system exists
Deep Learning Architectures Used	Transformers, RNNs, 3D-CNNs, Graph Convolutional Networks (GCNs)	Minimal use of modern deep learning; focus on classical methods
Key Example Frameworks	Transformer-based end-to-end continuous SLT by Camgoz et al. (2020)	No comparable PSL framework for continuous translation

Literature review

Aspect	Global Sign Language Translation (SLT) Research	Pakistan Sign Language (PSL) Research
Other Real-Time Models	Spatio-temporal GCN-based real-time SLT model by Zhu et al. (2021)	PSL systems do not attempt natural language generation
Hybrid Systems	RNN/CNN hybrids for continuous ASL video-to-text translation (2018)	Reliance on traditional image processing and sensors
Languages with Continuous Systems	Chinese Sign Language (CSL), Greek Sign Language, and others have datasets and translation systems	PSL lacks continuous datasets entirely
Datasets Availability	Multiple continuous sentence-level datasets available worldwide	Complete absence of sentence-level PSL datasets
Common Technologies	Vision-based deep neural models processing full sign sentences	Kinect sensors, Leap Motion, MediaPipe, template matching
ML Techniques	End-to-end neural translation with sequence modeling	SVM, PCA-KNN, DTW, template matching

Literature review

Aspect	Global Sign Language Translation (SLT) Research	Pakistan Sign Language (PSL) Research
Natural Language Output	Generation of spoken language glosses or full text sentences	Systems recognize only letters, numbers, or single gestures
Existing Translation Projects	Video-based SLT converting signs directly to text	Only text-to-PSL video playback (Abbas & Sarfraz, 2018)
Direction of Translation	Sign Language → Spoken/Natural Language text	PSL projects translate text → PSL, not PSL → Urdu
Confirmed Research Gap	Focus now on continuous SLT benchmarks	Reviews confirm absence of PSL continuous SLT
Current Status	Advanced global systems for real-time continuous SLT	No PSL-to-Urdu sentence-level translation system currently exists
Overall Conclusion	Continuous SLT widely researched and implemented	Significant unaddressed research gap in PSL translation

Scope

▶ **In-Scope**

- ▶ Real-time PSL recognition for selected vocabulary of 50+ sentences.
- ▶ Urdu text and speech output.
- ▶ Integration with Zoom (initial platform).
- ▶ Single-signer in controlled environments.
- ▶ Desktop-based.

Scope Cont...

▶ **Out-Of-Scope**

- ▶ Multi-signer recognition.
- ▶ Full PSL grammar support.
- ▶ Mobile/AR/VR deployment.
- ▶ Cloud-based processing.
- ▶ Reverse translation (Urdu to PSL).



System Architecture

► Layered Architecture:

- **User Interface Layer** – Configuration, subtitles, status.
- **Platform Integration Layer** – Zoom/Meet SDK, virtual camera/audio.
- **Computer Vision Layer** – MediaPipe for keypoint extraction.
- **Machine Learning Layer** – LSTM/Transformer for gesture recognition.
- **Translation & Synthesis Layer** – NLP, TTS, subtitle formatting.

► Data Flow:

Video → Keypoints → Gesture Recognition → Urdu Text → Subtitles + Speech

System Architecture Cont...

- ▶ **Core Modules:**

- ▶ **Video Capture Module** – Webcam input, frame buffering.
- ▶ **Pose Estimator** – MediaPipe Holistic for hand, face, body landmarks.
- ▶ **Gesture Recognizer** – LSTM/Transformer model for sign classification.
- ▶ **Translator** – Gloss-to-Urdu conversion with grammar correction.
- ▶ **TTS Engine** – Urdu speech synthesis (gTTS/pyttsx3).
- ▶ **Platform Integrator** – API connectors for Zoom/Meet.

Design Strategies

- ▶ **Modular & Layered Architecture** – Scalable, maintainable.
- ▶ **Object-Oriented Design (OOD)** – Reusability, separation of concerns.
- ▶ **Centralized Data Management** – Consistency, integrity.
- ▶ **Service-Oriented Components** – Reusable services (auth, validation).
- ▶ **Concurrency Control** – Safe multi-threading for real-time processing.

Technology Stack

► Frameworks & Libraries:

- MediaPipe, OpenCV, TensorFlow/PyTorch
- Zoom SDK, Google Meet API
- gTTS, pyttsx3
- Python 3.8+, Tkinter for UI



Technology Stack Cont...

- ▶ **Datasets:**

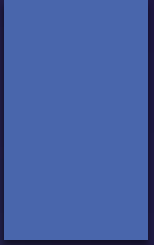
- ▶ Custom PSL dataset
- ▶ Multi-VSL dataset

- ▶ **Hardware:**

- ▶ Webcam (720p, 30 FPS)
- ▶ GPU recommended (GTX 1060+)



Expected Outcomes & Impact



- ▶ **Academic Contribution:**

- ▶ First continuous PSL sentence dataset.
- ▶ Novel hybrid ML model for PSL recognition.

- ▶ **Social Impact:**

- ▶ Bridges communication gap for Deaf community.
- ▶ Enhances accessibility in education, work, and social interactions.

- ▶ **Industrial Relevance:**

- ▶ Ready-to-use plugin for video conferencing.
- ▶ Scalable for future multilingual support.

Risks & Mitigation

▶ High-Risk Areas:

- ▶ **Latency >100 ms** → Optimize models, use GPU.
- ▶ **Platform API Changes** → Use adapter pattern, multi-platform support.
- ▶ **Low Recognition Accuracy** → Augment data, confidence thresholds.
- ▶ **Environmental Variability** → User guidelines, calibration tools.

▶ Volatile Areas:

- ▶ Vocabulary expansion, model updates, new platform integration.

System Architecture (High-Level) - Detailed Explanation

- Research Question Why & How**

- Answers**

Layered Architecture with Clear Separation of Concerns

Why This Pattern?

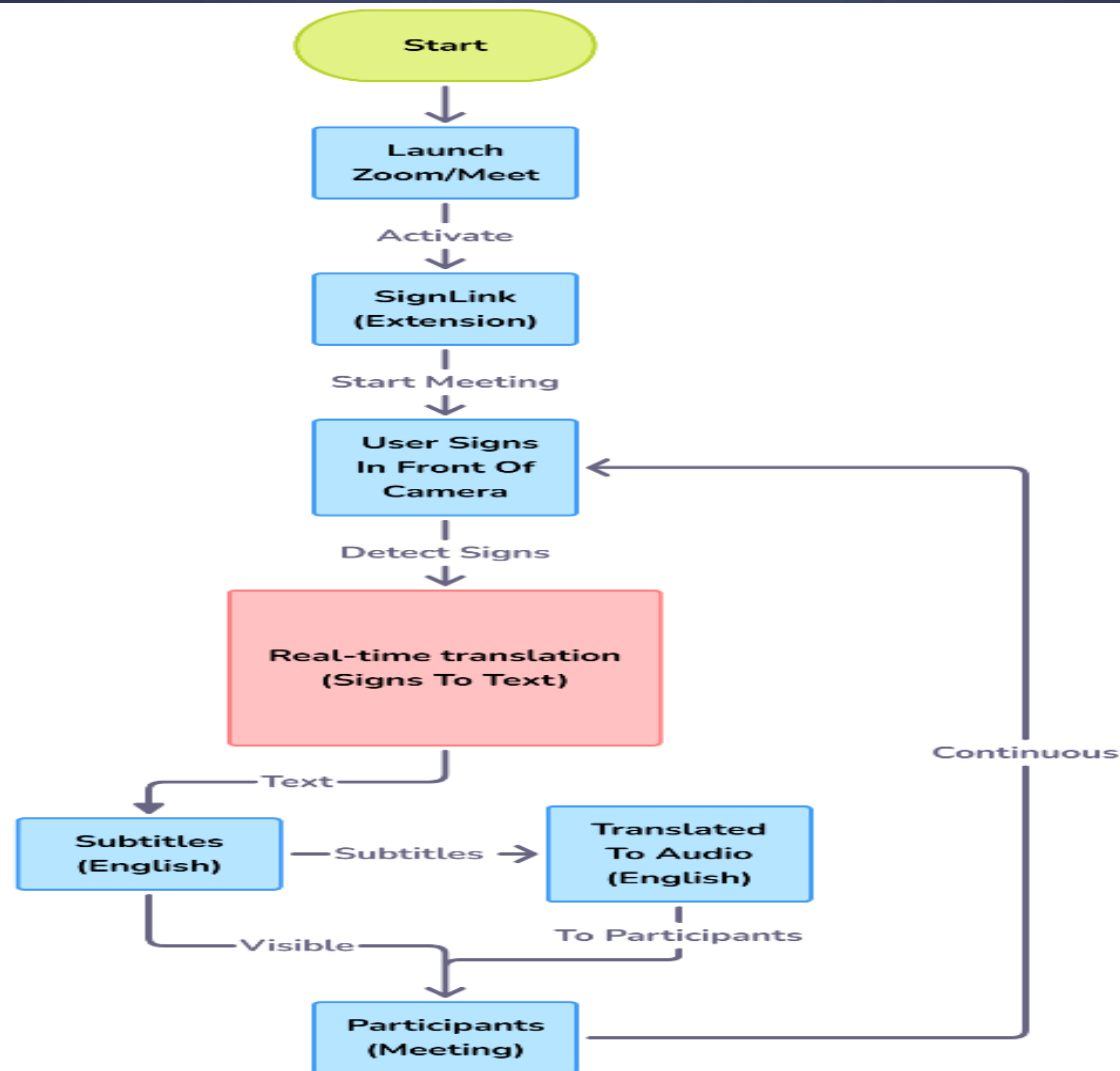
We selected a layered architecture to ensure maintainability, scalability, and clear separation of responsibilities. Each layer interacts only with adjacent layers through well-defined interfaces, making the system modular and testable.

Layer 1: User Interface Layer

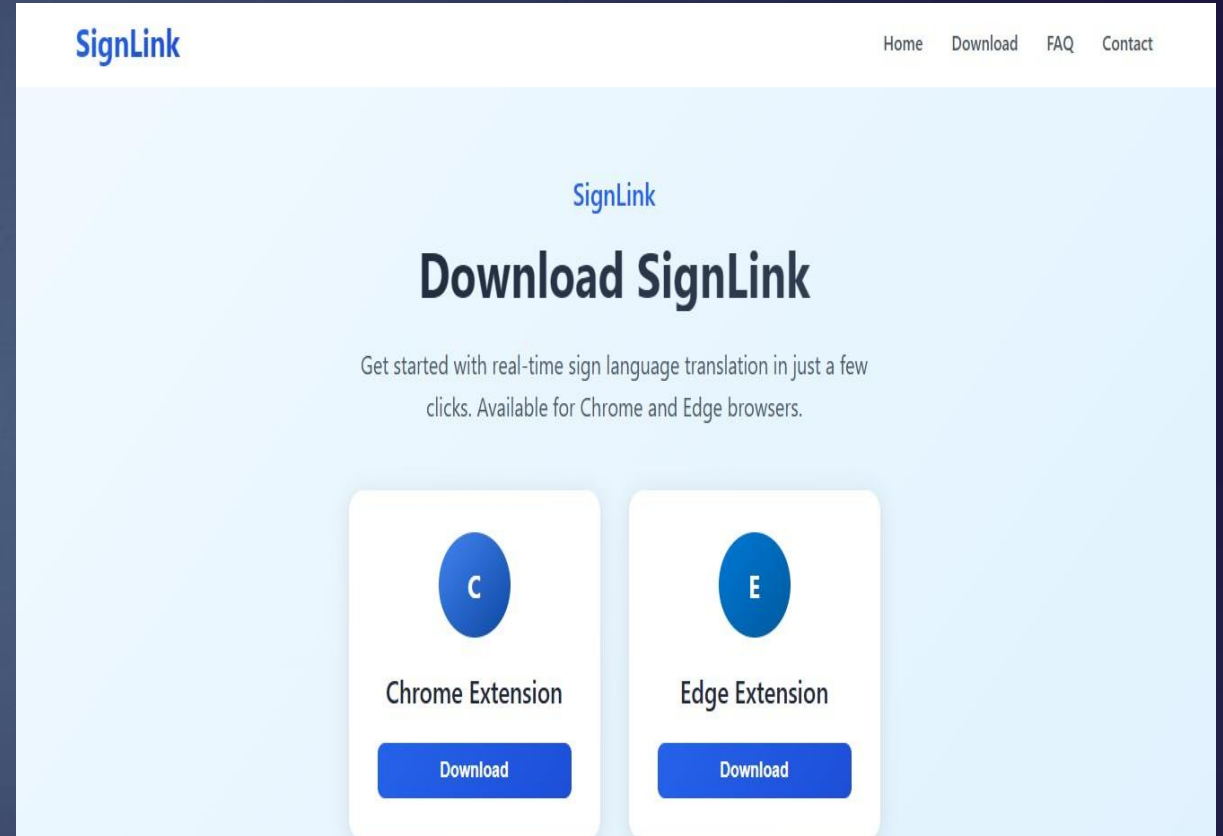
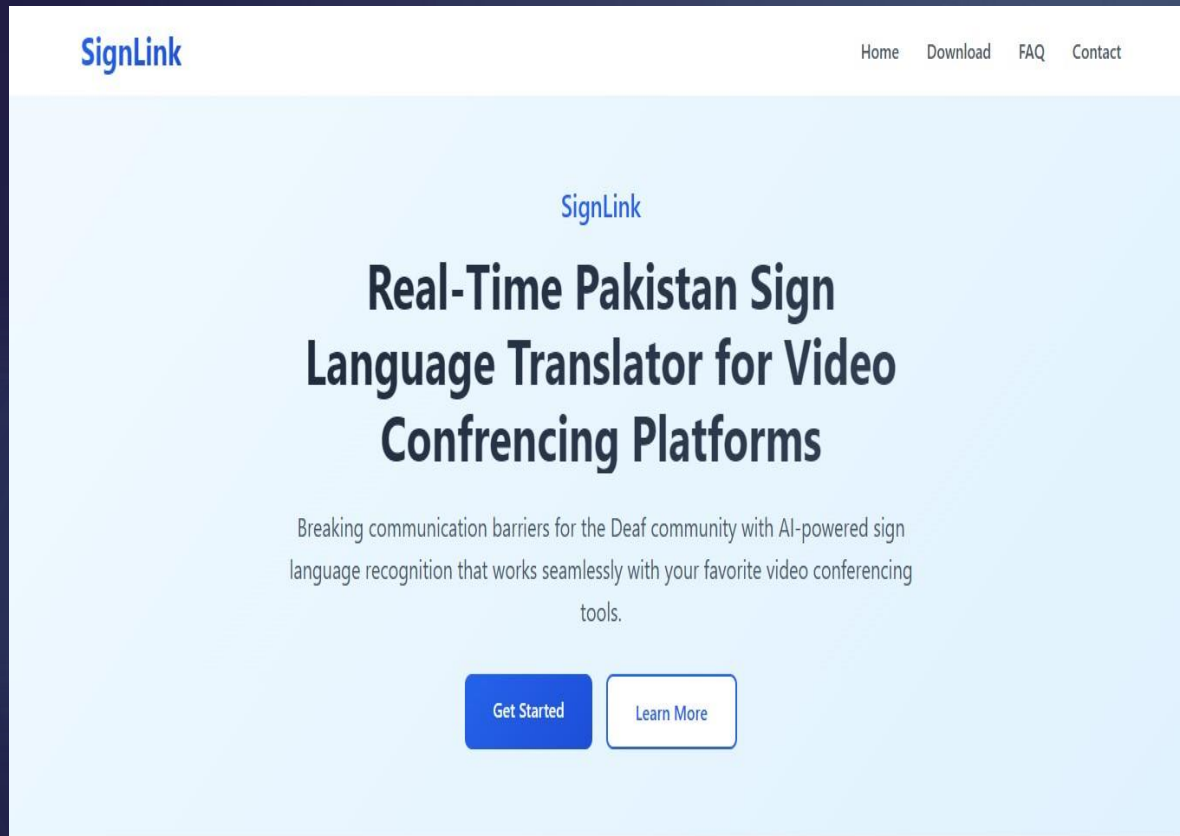
- ▶ **Purpose:** Bridge between user and system functionality.
- ▶ **Components:**
 - ▶ **Configuration Panel:** Settings for model selection, output preferences, platform choice.
 - ▶ **Status Monitor:** Real-time metrics display (FPS, confidence scores, latency).
 - ▶ **Subtitle Renderer:** On-screen text display with positioning and styling controls.
 - ▶ **Notification Service:** Alert system for errors and calibration requirements
- ▶ **Technical Implementation:**

Implements **Model-View-Controller (MVC)** pattern within this layer. The UI communicates with backend via **observer pattern** for real-time updates.

Cont...

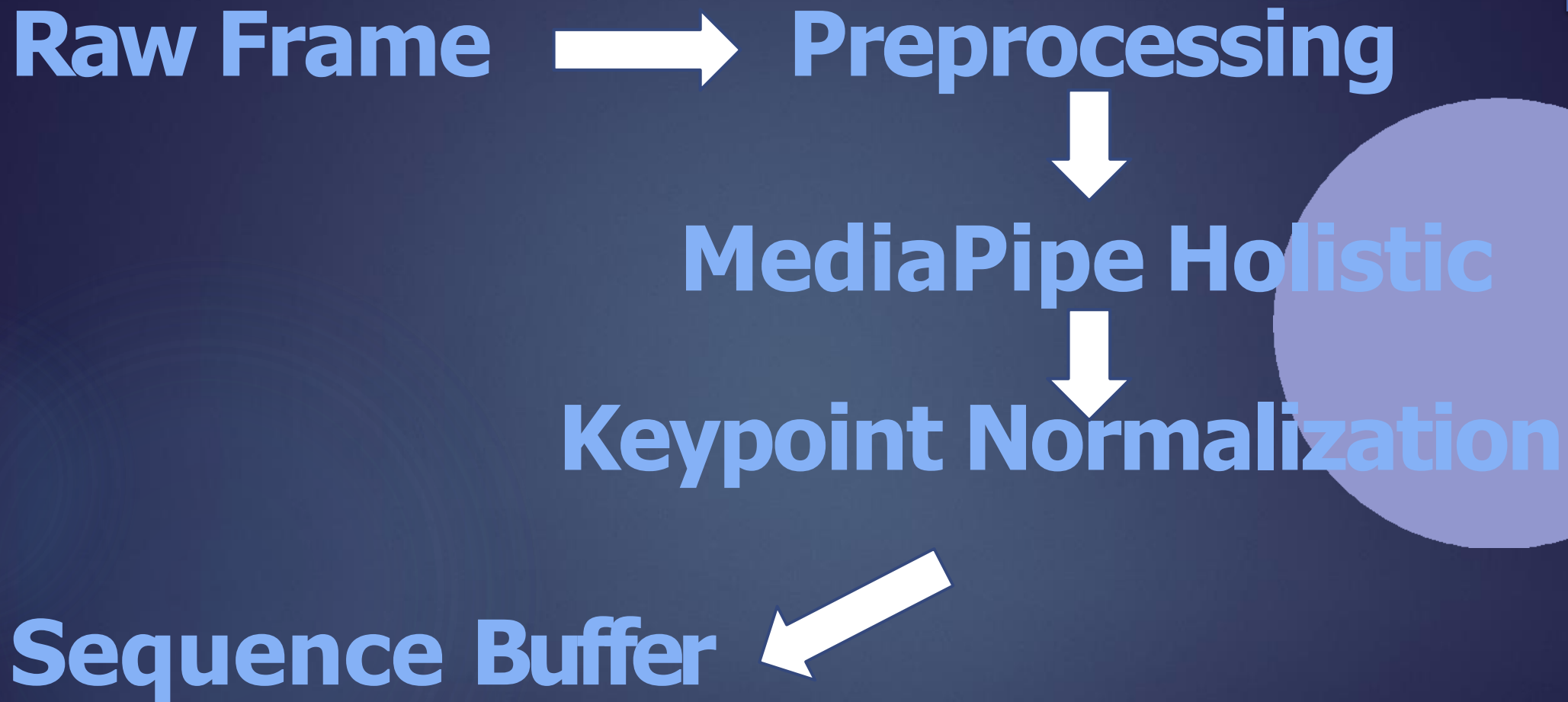


User Interface For Extension



Layer 2: Platform Integration Layer

- ▶ **Purpose:** Abstract platform-specific complexities.
- ▶ **Key Design Decision:** Adapter Pattern implementation
- ▶ **Components:**
 - ▶ Platform Adapter Factory: Detects platform and instantiates appropriate adapter.
 - ▶ Zoom Adapter: Uses Zoom SDK's VirtualCamera and VirtualSpeaker APIs.
 - ▶ Generic Platform Adapter: Fallback using virtual device drivers (OBS VirtualCam, VB-Audio).
 - ▶ Stream Router: Manages bidirectional video/audio streams using WebRTC protocol.



MediaPipe Integration

The screenshot displays a Visual Studio Code (VS Code) environment with the following components:

- File Explorer:** Shows a project structure with folders `signlink_env` and `signlink_ui`, and a `src` folder containing `main.py` and `pose_estimator.py`.
- Code Editor:** Displays the `main.py` file with the following Python code:

```
src > main.py > ...
1 import cv2
2 from pose_estimator
3
4 estimator = PoseEst
5 cap = cv2.VideoCapt
6
7 if not cap.isOpened
8     print("Error: C
9     exit()
10
11 print("Camera opene
12 frame_count = 0
13 print_landmarks = T
14
15 while cap.isOpened(
16     ret, frame = ca
17     if not ret:
18         print("Erro
19         break
20
21 # Extract landm
22 landmarks = est
```
- Terminal:** Shows the output of the program, displaying a list of 21 landmark coordinates in a 5x4 grid format:

```
0.36701229, 0.52029
-0.64971, 0.34847
1.57514942, -0.12859514, 0.22376606, 1.55425727, 0.13323209,
0.53605616, 2.3160727, -0.03399584, 0.22546193, 2.30301452,
0.52605128, 0.51646602, 2.96425891, 0.77895296, 0.22691064,
2.94592285, 1.37209022, 0.52755862, 3.05547214, 0.81654692,
0.22236709, 3.03215671, 1.44529831, 0.45175529, 3.17860246,
0.09656104, 0.25149012, 3.18085241, 0.68436944]]}
```
- SignLink Pose Estimation Window:** A video feed window showing a person's face and hand. Red dots represent detected landmarks, and white lines connect them to form a pose estimation model. The hand is shown in a gesture, and the face is also tracked.

The bottom status bar indicates the current file is `main.py`, line 10, column 1, using UTF-8 encoding and CRLF line endings. The Python version is 3.12.4 (signlink_env). The system tray shows the time as 8:25 PM on 1/7/2026, with a temperature of 9°C and clear weather.

Layer 3: Computer Vision Layer

- ▶ **Purpose:** Extract meaningful features from raw video
- ▶ **Technical Components:**
 - ▶ **Frame Preprocessor:** Resizes to 256×256, applies histogram equalization.
 - ▶ **MediaPipe Integrator:** Extracts 543 landmarks (21×2 hand + 33 pose + 468 face).
 - ▶ **Sequence Buffer:** Maintains temporal window of 30 frames (1-second context at 30 FPS).
 - ▶ **Data Normalizer:** Applies z-score normalization relative to shoulder width reference.
 - ▶ **Why MediaPipe?**

Provides real-time, on-device processing without cloud dependency. The holistic pipeline ensures synchronized extraction of hand, pose, and facial landmarks crucial for PSL grammar.

Layer 4: Machine Learning Layer

Purpose: Classify sign language gestures from temporal sequences

Model	Pros	Cons	Selected?
LSTM	Excellent temporal modeling, lower computational cost	May struggle with very long sequences	Primary
Transformer	Better long-range dependencies, parallel processing	Higher memory requirements	Alternative
CNN-LSTM Hybrid	Good spatial-temporal fusion	Complex training	Not selected

Cont...

► Implementation Details:

- **Input:** 30-frame sequences $\times$ 3 coordinates (x, y, z).
- **Architecture:** 3-layer BiLSTM with 256 units each, dropout (0.3).
- **Output:** Softmax over vocabulary (50+ signs initially).
- **Confidence Estimator:** Bayesian dropout for uncertainty quantification.

Dataset Specifications

▶ Custom PSL Dataset:

- ▶ **Size:** 50+ sentence types × 15 signers = 750+ samples
- ▶ **Format:** MP4 (H.264) + CSV annotations
- ▶ **Annotation:** Frame-level gloss labels + Urdu translations
- ▶ **Split:** 70% train, 15% validation, 15% test

▶ Multi-VSL Integration:

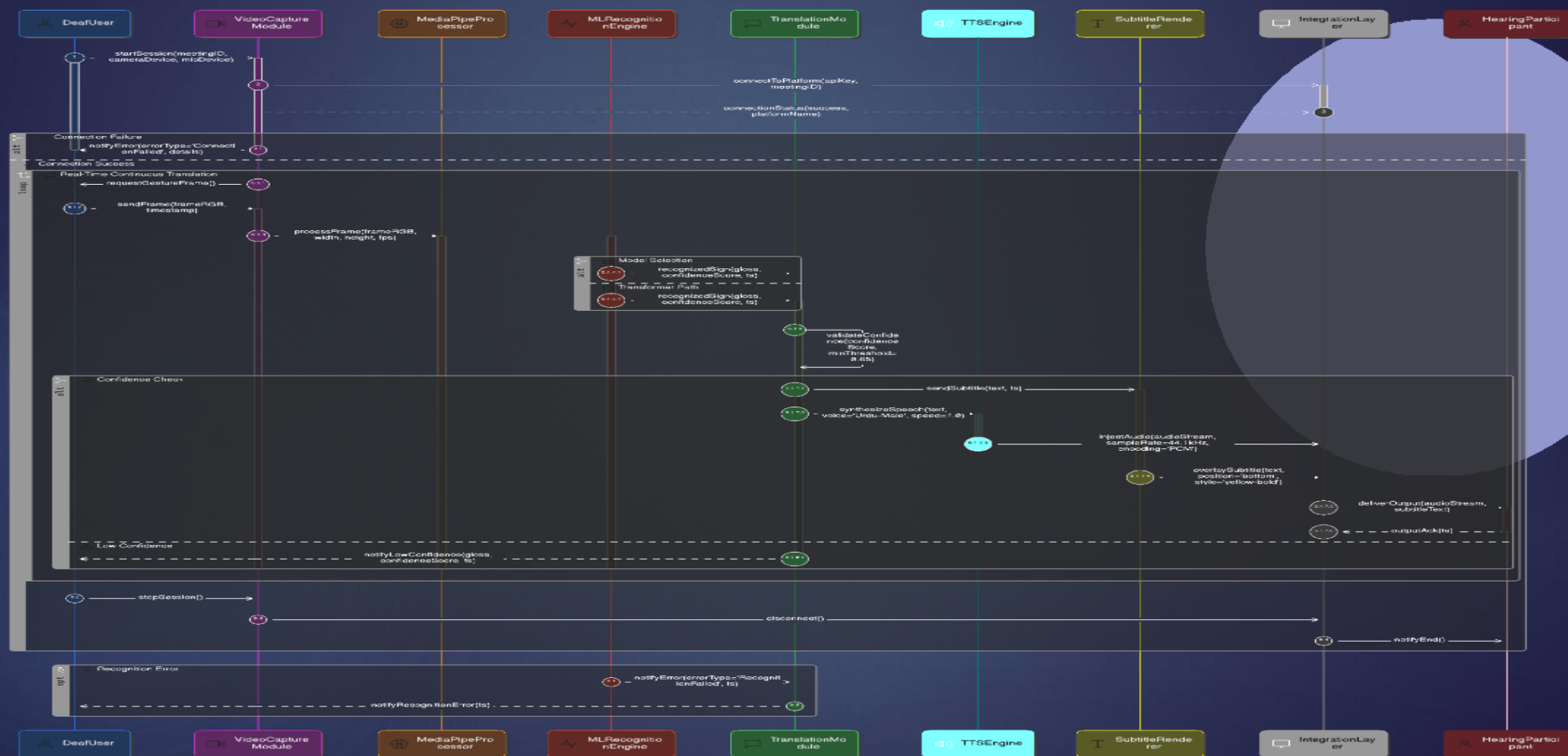
- ▶ Used for data augmentation through viewpoint transformation
- ▶ Helps model generalize to different signing styles



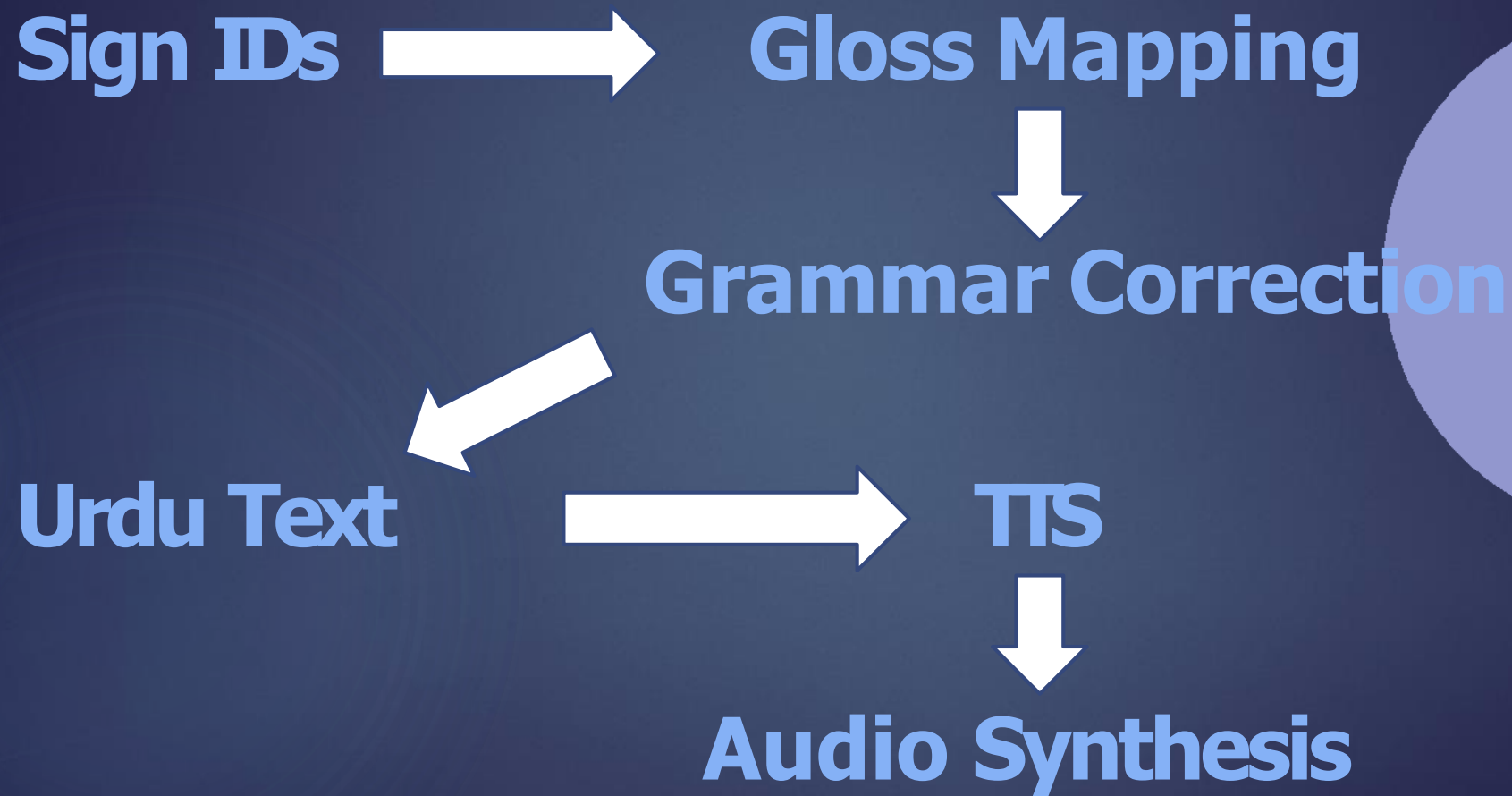
Layer 5: Translation & Synthesis

- ▶ **Purpose:** Convert gloss sequences to natural Urdu.
- ▶ **Technical Implementation:**
 - ▶ **Grammar Corrector:** Rule-based system addressing PSL-to-Urdu grammatical differences.
 - ▶ **TTS Engine:** gTTS with custom Urdu pronunciation dictionary.
 - ▶ **Subtitle Formatter:** Adds timing metadata using **SRT format** standards.

Data Flow Architecture



Cont...



Frameworks & Libraries Selection Rationale

Technology	Why Selected
MediaPipe	Real-time, cross-platform, no GPU requirement.
PyTorch	Dynamic computation graphs, better for research.
OpenCV	Industry standard, optimized video operations.
Zoom SDK	Official API, virtual device support.
gTTS	Free, good Urdu support.

Hardware Requirements

Justification

- ▶ **Minimum (CPU-only):**

- ▶ Intel i5-8th gen: AVX2 instructions for optimized MediaPipe
- ▶ 8GB RAM: 2GB OS + 3GB Python + 3GB model weights

- ▶ **Recommended (GPU-accelerated):**

- ▶ NVIDIA GTX 1060: 1280 CUDA cores, 6GB VRAM
- ▶ Enables batch processing and lower latency

- ▶ **Webcam Specifications:**

- ▶ 720p @ 30FPS: Balance between detail and processing load



Expected Outcomes & Impact



- ▶ **Academic Contribution:**

- ▶ First continuous PSL sentence dataset.
- ▶ Novel hybrid ML model for PSL recognition.

- ▶ **Social Impact:**

- ▶ Bridges communication gap for Deaf community.
- ▶ Enhances accessibility in education, work, and social interactions.

- ▶ **Industrial Relevance:**

- ▶ Ready-to-use plugin for video conferencing.
- ▶ Scalable for future multilingual support.

Future Roadmap

- ▶ **Short-term:** User testing with Deaf community partners.
- ▶ **Medium-term:** Vocabulary expansion to 200+ signs.
- ▶ **Long-term:** Mobile optimization & regional dialect support.
- ▶ **Vision:** Integration with educational platforms and public service kiosks.

Conclusion

► **Bridging Communication Through Innovation**

► **Core Achievement:**

- We have designed and architected SignLink—a pioneering real-time Pakistan Sign Language translation system that transforms accessibility in digital communication. By leveraging AI-driven gesture recognition, real-time Urdu synthesis, and seamless platform integration, we have created a working prototype that demonstrates the technical feasibility and social impact of automated sign language translation.

Thank You

For Questions & Discussion

Project Team:

Muhammad Hassan Shahbaz | Behlol Abbas

Advisor:

Dr. Fahad Maqbool

Project Manager:

Dr. Muhammad Ilyas

Repository: <https://github.com/devHassan007/FYP-PSLTS>

